Proxy inverso y balanceador de carga

Apache, Nginx y HAProxy

📧 José Domingo Muñoz

🏫 IES Gonzalo Nazareno · Dos Hermanas

📚 SRI · Servicios de Red e Internet

01

Proxy inverso

Concepto y problemas comunes

¿Qué es un proxy inverso?

> *Un **proxy inverso** acepta las peticiones de los clientes, las **reenvía** a uno o varios servidores internos (**backends**) y devuelve sus respuestas como si las hubiera generado él mismo.*

- El cliente **no accede directamente** a los backends: solo ve al proxy
- Los backends pueden estar en una **red privada** sin acceso desde fuera
- Para el backend, **el cliente es el proxy**: la conexión TCP la abre el proxy

ℹ️ Un **proxy** actúa en nombre de los clientes; un **proxy inverso**, en nombre de los servidores (lo vimos en *Protocolo HTTP*).

Casos de uso

PROTECCIÓN Y CENTRALIZACIÓN

- **Ocultar** la infraestructura backend
- Publicar varias aplicaciones internas bajo **un único nombre o IP**
- Aplicar **filtros** y **autenticación** en un solo punto
- Centralizar las **cabeceras** de seguridad

RENDIMIENTO Y OPERACIÓN

- **Distribuir carga** entre varios backends
- **Terminación TLS**: HTTPS solo en el proxy
- **Caché** de respuestas estáticas
- Cambiar los backends **sin que el cliente lo note**

Lo que tiene que resolver el proxy

El proxy hace una **petición nueva** al backend y devuelve su respuesta: a veces tiene que **cambiar cabeceras**.

`HOST` (EN LA PETICIÓN)

Por defecto llega el nombre del **destino del proxy** (`interno.example1.org`). Si el backend tiene **virtual hosts por nombre**, puede responder otro sitio.

→ El proxy debe mandar el **nombre que espera el backend**.

`LOCATION` (EN LA RESPUESTA)

Una redirección del backend lleva **su nombre interno** (`Location: http://interno.example1.org/nuevo`), que no se resuelve desde fuera.

→ El proxy debe **reescribirla** con el nombre público.

⚠️ **Publicar en una subruta** (`www.ejemplo.org/shop/`): la aplicación cree que está en la raíz, así que sus rutas absolutas (`/css/estilo.css`, enlaces, redirecciones) se piden al proxy **sin** `/shop` y la página sale rota.

Problema con los enlaces HTML

Las páginas y aplicaciones web tienen que **construir bien sus enlaces** para funcionar detrás de un proxy inverso. Publicada en `www.ejemplo.org/shop/`:

ENLACE EN LA PÁGINA DEL BACKEND	¿FUNCIONA?
`https://interno.example1.org/css/estilo.css`	❌ El cliente no llega al nombre interno
`/css/estilo.css`	❌ Se pide fuera de `/shop/` (sin subruta, sí funciona)
`css/estilo.css`	✅ Se pide dentro de `/shop/`

ℹ️ El proxy puede **reescribir el HTML** al pasar (`mod_proxy_html` en Apache, `sub_filter` en Nginx), pero no cambia los enlaces que genera el JavaScript.

02

Proxy inverso con Apache

mod_proxy y ProxyPass

Activar los módulos necesarios

Apache implementa el proxy inverso con los módulos `mod_proxy` y `mod_proxy_http`.

```
sudo a2enmod proxy proxy_http
sudo systemctl restart apache2
```

OTROS MÓDULOS RELACIONADOS

- `mod_headers` — añadir o cambiar cabeceras de la petición y de la respuesta
- `mod_proxy_html` — reescribir los enlaces del HTML que devuelve el backend

Configuración básica

Un virtual host por cada nombre público (`www.app1.org`, `www.app2.org`...):

```
<VirtualHost *:80>
    ServerName www.app1.org
    ProxyPass        "/" "http://interno.example1.org/"
    ProxyPassReverse "/" "http://interno.example1.org/"
</VirtualHost>
```

- **`ProxyPass`** — reenvía las peticiones al backend
- **`ProxyPassReverse`** — reescribe `Location` en las respuestas (redirecciones)
- El proxy tiene que **resolver** `interno.example1.org` (DNS o `/etc/hosts`)

El `Host` y las redirecciones en Apache

`PROXYPRESERVEHOST`

- **`Off`** (por defecto): el backend recibe el nombre de `ProxyPass` (`interno.example1.org`)
- **`On`**: recibe el nombre que pidió el cliente (`www.app1.org`)

```
ProxyPreserveHost On
```

`PROXYPASSREVERSE`

Cambia en la respuesta

`http://interno.example1.org/nuevodirectorio`

por

`http://www.app1.org/nuevodirectorio`

Sin esta línea, el cliente recibe el nombre interno.

Proxy de subrutas en Apache

```
ProxyPass        "/shop/" "http://192.168.100.10:3000/"
ProxyPassReverse "/shop/" "http://192.168.100.10:3000/"

ProxyPass        "/game/" "http://192.168.100.10:8080/"
ProxyPassReverse "/game/" "http://192.168.100.10:8080/"

# /shop y /game sin barra no coinciden con ProxyPass
RedirectMatch "^/(shop|game)$" "/$1/"
```

- Las dos URL de `ProxyPass` tienen que acabar **igual**: Apache cambia un prefijo por el otro tal cual (con `"/shop/" "http://…:3000"`, `/shop/a` iría a `http://…:3000a`)
- `/shop` (sin barra) no coincide con `/shop/`: sin el `RedirectMatch`, lo sirve el `DocumentRoot` del proxy y da **404**

Cabeceras hacia el backend

Para el backend, el cliente es el proxy. `mod_proxy_http` añade **automáticamente**:

CABECERA	CONTENIDO
`X-Forwarded-For`	IP del cliente original (se van acumulando si hay varios proxies)
`X-Forwarded-Host`	`Host` que pidió el cliente
`X-Forwarded-Server`	`ServerName` del proxy

Con ellas el backend sabe **quién es el cliente real**: por ejemplo, para registrar su IP en el log en lugar de la del proxy.

Añadir más cabeceras

Si hay que mandar **más cabeceras** al backend, se usa el módulo **`mod_headers`** (`sudo a2enmod headers`) con la directiva `RequestHeader`:

```
RequestHeader set X-Real-IP         "expr=%{REMOTE_ADDR}"
RequestHeader set X-Forwarded-Proto "http"
```

- **`X-Real-IP`**: la IP del cliente, sola (sin la lista que acumula `X-Forwarded-For`)
- **`X-Forwarded-Proto`**: si el cliente llegó por `http` o `https`, para que el backend genere enlaces y redirecciones con el protocolo correcto

03

Proxy inverso con Nginx

proxy_pass y proxy_params

Configuración básica

Nginx incluye el proxy inverso de forma **nativa**, sin activar módulos.

```
server {
    listen 80;
    server_name www.app1.org;

    location / {
        proxy_pass http://interno.example1.org/;
        include proxy_params;
    }
}
```

- `proxy_pass` — destino al que se reenvía la petición
- `include proxy_params` — incluye el fichero donde se definen las **cabeceras que se envían al backend**
- Si Nginx **no resuelve** el nombre de `proxy_pass`, no arranca (`host not found in upstream`)

El archivo `proxy_params`

`/etc/nginx/proxy_params` (Debian). Sin él, Nginx **no** añade estas cabeceras y manda `Host: $proxy_host` (el nombre de `proxy_pass`).

```
proxy_set_header Host              $http_host;                 # nombre que pidió el cliente (www.app1.org)
proxy_set_header X-Real-IP         $remote_addr;               # IP del cliente
proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for; # lista que traía + IP del cliente
proxy_set_header X-Forwarded-Proto $scheme;                    # http o https, según llegó el cliente
```

⚠️ Si el backend tiene virtual hosts con los **nombres internos**, no uses `include proxy_params`: pon solo las cabeceras que necesites con `proxy_set_header`.

Reescritura de redirecciones

Nginx **ya reescribe** la cabecera `Location` por defecto (`proxy_redirect default`): cambia la URL de `proxy_pass` por la de la `location` y la completa con el nombre que pidió el cliente.

```
location / {
    proxy_pass     http://interno.example1.org/;
    # proxy_redirect default;   ← es el comportamiento por defecto
    # proxy_redirect off;       ← no reescribe: el cliente recibe el nombre interno
}
```

RESPUESTA DEL BACKEND	LO QUE RECIBE EL CLIENTE
`Location: http://interno.example1.org/nuevodirectorio`	`Location: http://www.app1.org/nuevodirectorio`

Equivale al `ProxyPassReverse` de Apache, que en cambio hay que escribir siempre.

Proxy de subrutas en Nginx

```
server {
    listen 80;
    server_name www.ejemplo.org;

    location /shop/ {
        proxy_pass http://192.168.100.10:3000/;
    }
    location /game/ {
        proxy_pass http://192.168.100.10:8080/;
    }
}
```

- La **barra final** de `proxy_pass` decide qué llega al backend: con `/`, se quita el prefijo, como en Apache (`/shop/a` → `/a`); sin barra, llega la URL entera (`/shop/a`)
- `/shop` (sin barra) no coincide con `/shop/`, pero como la `location` acaba en `/`, Nginx **redirige solo** a `/shop/`: no hace falta el `RedirectMatch`

04

Balanceo de carga

Algoritmos y software

Algoritmos de balanceo — estáticos

No tienen en cuenta el estado de los servidores: el reparto se decide **de antemano**.

ROUND ROBIN

Las peticiones se reparten **por turnos** entre los servidores. Requiere servidores equivalentes.

ROUND ROBIN PONDERADO

Cada servidor tiene un **peso**: los más potentes reciben más peticiones.

STICKY (AFINIDAD)

Un mismo cliente va **siempre al mismo servidor** (por su IP o con una cookie).

HASH

Una función *hash* sobre la IP, la URL... **determina** qué servidor atiende cada petición.

Algoritmos de balanceo — dinámicos

Tienen en cuenta el **estado real** de los servidores en cada momento.

CONEXIONES MÍNIMAS

La nueva petición va al servidor con **menos conexiones activas**.

MENOR TIEMPO DE RESPUESTA

Se elige el servidor con el **tiempo de respuesta más bajo** según las últimas mediciones.

ℹ️ Se adaptan mejor a peticiones de duración muy distinta, pero el balanceador tiene que **medir** el estado de cada servidor.

Software de balanceo de carga

SOFTWARE	CARACTERÍSTICAS
HAProxy	Especializado en balanceo, TCP y HTTP. *Health checks* activos y página de estadísticas
Nginx (`upstream`)	Servidor web que también balancea: bloque `upstream` + `proxy_pass`
Apache (`mod_proxy_balancer`)	`<Proxy "balancer://...">` con `BalancerMember`
Traefik	Proxy inverso para **contenedores** (Docker, Kubernetes): se configura solo a partir de etiquetas

ℹ️ También hay balanceadores **hardware** y los que ofrecen los proveedores de nube. En esta unidad usamos **HAProxy**.

05

Balanceador de carga con HAProxy

Distribución de tráfico entre backends

¿Qué es HAProxy?

> *HAProxy (High Availability Proxy) es **software libre de alto rendimiento** especializado en **balanceo de carga** y **proxy inverso** para servicios TCP y HTTP.*

- **Estabilidad** y rendimiento muy contrastados
- **Health checks** automáticos sobre los backends
- Varios **algoritmos** de balanceo
- Puede balancear **TCP genérico** o entender **HTTP** (cabeceras, cookies)
- Página de **estadísticas** para ver y operar el balanceador

Escenario de laboratorio

MÁQUINA	ROL	RED EXTERNA	RED DE DATOS
balanceador	HAProxy	`192.168.10.10`	`192.168.100.1`
apache1	Servidor web	`192.168.10.11`	`192.168.100.100`
apache2	Servidor web	`192.168.10.12`	`192.168.100.101`
Tu equipo	Cliente	`192.168.10.1`	—

- El cliente accede a `www.example.org`, que apunta a la IP **externa del balanceador**
- HAProxy se conecta a los servidores web por la **red de datos**
- Cada servidor web muestra su nombre, para ver quién atiende cada petición

Instalación

```
sudo apt install haproxy
systemctl status haproxy

# Validar la sintaxis antes de aplicar cambios
sudo haproxy -c -f /etc/haproxy/haproxy.cfg

# Aplicar cambios
sudo systemctl reload haproxy
```

Los mensajes de HAProxy van al journal: `journalctl -u haproxy`.

Estructura del archivo de configuración

`/etc/haproxy/haproxy.cfg` se organiza en secciones:

SECCIÓN	PARA QUÉ SIRVE
`global`	Parámetros del demonio (logs, usuario, límites)
`defaults`	Valores por defecto para `frontend`, `backend` y `listen`
`frontend`	Cómo se **reciben** las peticiones (IP, puerto)
`backend`	A qué **servidores** se reenvían y con qué algoritmo
`listen`	`frontend` + `backend` en un solo bloque

ℹ️ El fichero que instala Debian ya trae `global` y `defaults`: solo hay que añadir el `frontend` y el `backend`.

Secciones `global` y `defaults`

```
global
    log /dev/log local0
    user haproxy
    group haproxy

defaults
    log     global
    mode    http
    option  httplog
    timeout connect 5s
    timeout client  50s
    timeout server  50s
```

- **`mode http`** — entiende HTTP (alternativa: `mode tcp` para TCP genérico)
- **`timeout`** — tiempos de espera de conexión, cliente y servidor

Secciones `frontend` y `backend`

```
frontend servidores_web
    bind *:80
    default_backend servidores_web_backend

backend servidores_web_backend
    balance roundrobin
    option  httpchk GET /
    server  apache1 192.168.100.100:80 check
    server  apache2 192.168.100.101:80 check
```

- **`bind *:80`** — escucha en el puerto 80 · **`default_backend`** — adónde van las peticiones
- **`balance`** — algoritmo de balanceo · **`server ... check`** — con *health check* activo

Algoritmos en HAProxy

BALANCE	ALGORITMO	TIPO
`roundrobin`	Round Robin (ponderado con `weight`)	Estático
`source`	Hash de la IP del cliente: afinidad por IP	Estático
`uri`	Hash de la URL: la misma URL, al mismo servidor	Estático
`leastconn`	Conexiones mínimas	Dinámico

```
server apache1 192.168.100.100:80 check weight 2
server apache2 192.168.100.101:80 check weight 1
```

Health checks

HAProxy pide `GET /` a cada servidor cada cierto tiempo (vale un 2xx o 3xx):

```
backend servidores_web_backend
    option httpchk GET /
    server apache1 192.168.100.100:80 check inter 2s fall 3 rise 2
    server apache2 192.168.100.101:80 check inter 2s fall 3 rise 2
```

PARÁMETRO	SIGNIFICADO
`inter`	Intervalo entre comprobaciones
`fall`	Fallos seguidos para marcarlo **DOWN**
`rise`	Aciertos seguidos para volver a marcarlo **UP**

Si uno falla, **deja de enviarle tráfico**. Si caen todos: **503**.

Acceso desde el cliente

En el `/etc/hosts` del cliente: `192.168.10.10 www.example.org`

Varias peticiones seguidas:

```
$ for i in 1 2 3; do curl -s http://www.example.org/ | grep Servidor; done
<h1>Servidor: apache1</h1>
<h1>Servidor: apache2</h1>
<h1>Servidor: apache1</h1>
```

Con `roundrobin`, las respuestas **alternan** entre los dos servidores.

La IP del cliente en los servidores web

Para el servidor web, el cliente es el **balanceador**:

```
192.168.100.1 - - [03/Oct/2026:10:15:02 +0200] "GET / HTTP/1.1" 200 312 "-" "curl/8.14.1"
```

Con `option forwardfor`, HAProxy añade la cabecera `X-Forwarded-For` con la IP real del cliente:

```
backend servidores_web_backend
    option forwardfor
```

ℹ️ La cabecera llega al servidor web, pero el log **no la registra** hasta que se cambia su formato.

Registrar la IP real en el log

```
# Apache
LogFormat "%{X-Forwarded-For}i %l %u %t \"%r\" %>s %O" balanceado
CustomLog ${APACHE_LOG_DIR}/access.log balanceado
```

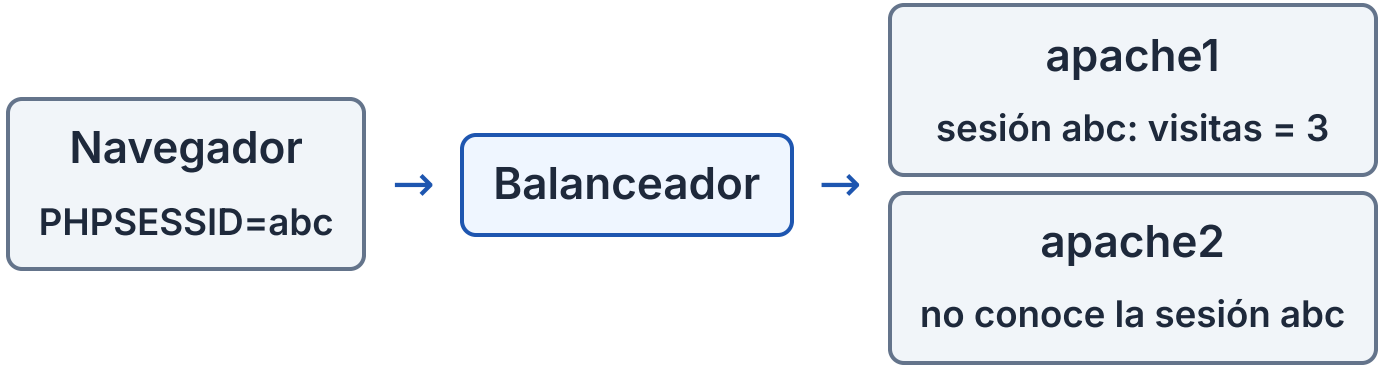
```
# Nginx
log_format balanceado '$http_x_forwarded_for - [$time_local] "$request" $status';
access_log /var/log/nginx/access.log balanceado;
```

Resultado:

```
192.168.10.1 - - [03/Oct/2026:10:16:40 +0200] "GET / HTTP/1.1" 200 312
```

Afinidad de sesión: el problema

Una **sesión** de PHP se guarda **en el servidor** (`/var/lib/php/sessions`); el navegador solo guarda la cookie con el identificador (`PHPSESSID`).



Con `roundrobin`, la siguiente petición va a `apache2`, que **no tiene** esa sesión: crea una nueva y el contador vuelve a empezar.

Afinidad de sesión con una cookie

HAProxy añade su **propia cookie** con el servidor que ha atendido al cliente:

```
backend servidores_web_backend
    balance roundrobin
    cookie  SERVERID insert
    server  apache1 192.168.100.100:80 check cookie s1
    server  apache2 192.168.100.101:80 check cookie s2
```

1. Primera petición, sin cookie: `roundrobin` elige `apache1` y HAProxy añade a la respuesta `Set-Cookie: SERVERID=s1`
2. El navegador envía `SERVERID=s1` en cada petición y HAProxy la manda **siempre a** **`apache1`**
3. **`cookie s1`** en cada `server` es el valor que lo identifica en la cookie

ℹ️ Opciones habituales: `cookie SERVERID insert indirect nocache`. **`indirect`**: HAProxy usa `SERVERID` para elegir el servidor y la **quita** antes de pasarle la petición, así el servidor web solo recibe sus cookies (`PHPSESSID`). **`nocache`**: que ninguna caché intermedia guarde esa respuesta, o mandaría a todos al mismo servidor.

Comprobar la afinidad

`curl` **no guarda cookies**: cada petición es un cliente nuevo y el contador siempre vale 1.

```
$ curl -sI http://www.example.org/sesion.php | grep -i set-cookie
set-cookie: PHPSESSID=k3j9q2...; path=/
set-cookie: SERVERID=s1; path=/
```

Para que `curl` guarde las cookies y las envíe en las siguientes peticiones:

```
$ curl -s -c cookies.txt -b cookies.txt http://www.example.org/sesion.php
```

Repetido varias veces: **siempre el mismo servidor** y el contador sube. El navegador lo hace solo.

Otras formas de afinidad: `balance source` (por IP) o sesiones **compartidas** entre servidores (base de datos, Redis).

Página de estadísticas

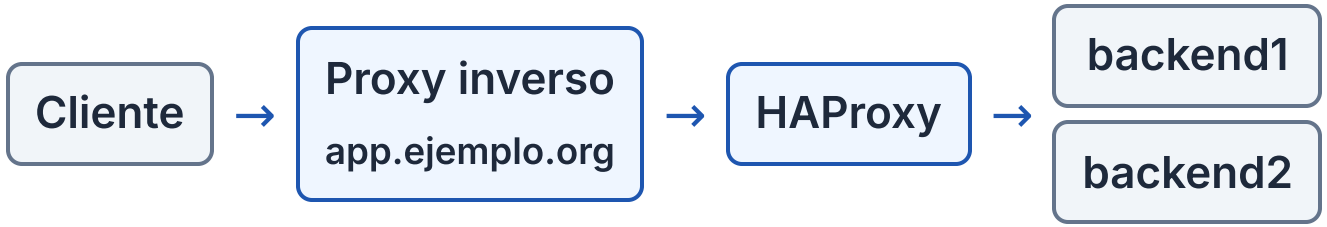
Bloque `listen` propio, en otro puerto (`http://192.168.10.10:8404/stats`):

```
listen estadisticas
    bind *:8404
    stats enable
    stats uri   /stats
    stats auth  admin:asir
    stats admin if TRUE
```

- Estado de cada servidor (**UP** / **DOWN** / **MAINT**), sesiones y errores
- Con `stats admin`, poner un servidor en **mantenimiento** desde la web

⚠️ Siempre con autenticación y nunca accesible desde redes públicas.

Proxy inverso y balanceador juntos



- **Proxy inverso**: único punto de entrada; elige el destino por el **nombre** o la **ruta**
- **Balanceador**: reparte entre servidores **que sirven lo mismo** y retira los que fallan
- Para el proxy, HAProxy es **un backend más**; para HAProxy, el cliente es el proxy
- `X-Forwarded-For` **acumula** las IP: `X-Forwarded-For: <cliente>, <proxy>`

Para profundizar

- Documentación oficial de HAProxy
- Apache mod_proxy
- Nginx — proxy_pass

SRI · UNIDAD 2 — PROXY INVERSO Y BALANCEADOR DE CARGA

¡Gracias!

Proxy inverso y balanceo

📧 José Domingo Muñoz 🏫 IES Gonzalo Nazareno · Dos Hermanas 📚 SRI